

synthesis

Relatório técnico do frontend

Estado atual da aplicação web, integração com a API Django, fluxos por perfil, regras editoriais e cobertura de validação do MVP.

Atualizado em 14/09/2026.

1. Resumo executivo

O frontend **synthesis** está implementado em Next.js e conectado à API Django. O produto organiza a coleta semanal de relatos, a curadoria feita pela gerente e a publicação de versões imutáveis do relatório executivo em PDF. A administração funcional também está disponível na própria aplicação.

- gestores criam relatos guiados, enviam fotos e editam o próprio conteúdo enquanto o ciclo permite;
- a gerente controla ciclos, acompanha pendências e seleciona o material da semana;
- o editor cria uma camada editorial sem alterar o relato original;
- a primeira página sintetiza leitura, indicadores, destaques, atenções e prioridades;
- cada publicação cria uma nova versão de PDF, preservando as anteriores;
- administradores operam cadastros e consultam registros e auditoria.

Separação essencial: `ActivityReport` é a fonte da verdade enviada pelo gestor. `ReportCard` é o snapshot editorial usado no relatório. Editar um card não altera o

relato original.

2. Stack e execução

Aplicação

Next.js 16.3, React 19.3 e TypeScript 5.9 com App Router.

Interface

Tailwind CSS, componentes locais, Radix Dialog e ícones Lucide.

Formulários

React Hook Form e Zod para estado e validação.

Dados

TanStack Query sobre um contrato único `SynthesisApi`.

Edição

dnd-kit para ordenação e react-easy-crop para imagens.

Qualidade

ESLint, TypeScript, Vitest e Playwright.

Desenvolvimento local

```
cp .env.example .env.local
npm ci
npm run dev
```

O front abre em `http://localhost:3000` e, em desenvolvimento, usa por padrão `http://localhost:8000/api`. Em produção, usa `/api` na mesma origem. A configuração `NEXT_PUBLIC_API_URL=mock` existe somente para execução isolada e testes.

3. Arquitetura do frontend

<code>src/</code>	
<code>app/</code>	páginas e rotas do App Router
<code>components/activity/</code>	cadastro, edição, modelos e fotos dos relatos
<code>components/report/</code>	editor, validação e prévia executiva/A4
<code>components/admin/</code>	CRUD administrativo orientado por metadados
<code>components/layout/</code>	shell, cabeçalho e navegação por perfil
<code>services/api.ts</code>	contrato, cliente HTTP e implementação mock
<code>services/auth.ts</code>	sessão, login, logout e CSRF
<code>services/queries.ts</code>	consultas compartilhadas do TanStack Query

services/administration.ts	cliente da API administrativa
lib/	ciclos, PDF e utilitários
types.ts	tipos centrais do domínio
test/	testes unitários e de componentes
e2e/	jornadas Playwright mockada e integrada

Componentes de tela consomem `api` ; a escolha entre `httpApi` e `mockApi` ocorre em um único ponto. O cliente HTTP centraliza URL base, cookies, CSRF, paginação, serialização e tradução dos erros do backend.

4. Autenticação e segurança

- login por e-mail e senha, com sessão Django em cookie `HttpOnly` ;
- requisições usam `credentials: include` e token CSRF em operações mutáveis;
- a sessão é restaurada por `/me` após recarga;
- logout invalida leituras pendentes, limpa tokens legados e é propagado entre abas;
- respostas 401 limpam o estado local e redirecionam o fluxo de autenticação;
- arquivos PDF são privados e dependem de sessão válida.

`RoleGuard` e a navegação por perfil melhoram a experiência, mas a autorização real permanece no backend. O E2E integrado verifica negação de objetos alheios, endpoints por perfil e PDF anônimo.

5. Fluxos por perfil

Perfil	Responsabilidades no frontend	Rotas principais
Gestor	Lista, cria e consulta relatos; edita apenas os próprios durante ciclo aberto ou reaberto.	<code>/relatos</code> , <code>/relatos/novo</code> , <code>/relatos/[id]</code>
Gerente	Controla ciclos, seleciona relatos, edita o briefing, valida pendências, publica PDFs e consulta histórico.	<code>/ciclos</code> , <code>/relatorio-semanal</code> , <code>/historico</code>
Admin	Gerencia cadastros e consulta dados operacionais, arquivos e auditoria conforme capacidades da API.	<code>/administracao</code>

Relatos guiados

O cadastro começa pela escolha entre ação/evento, entrega/marco e atendimento/articulação. Perguntas, exemplos e instruções mudam por modelo, mas o payload editorial permanece estável. Há contadores, prévia do card, upload de foto principal e fotos adicionais. Evidência e próximo passo são opcionais; notas internas ficam no relato original e não são copiadas para o PDF.

Ciclos semanais

Em `/ciclos` , a gerente abre o próximo período, ajusta o prazo do ciclo ativo, encerra a coleta ou reabre um ciclo encerrado com justificativa. Não é permitido manter dois ciclos ativos simultaneamente.

6. Relatório executivo

1. A gerente seleciona os relatos em `/relatorio-semanal/selecao` .
2. A API cria o `WeeklyReport` , seções por área e snapshots `ReportCard` .
3. O editor permite alterar textos editoriais, foto, classificação, pedido de decisão, responsável e prazo.
4. Os cards podem ser reordenados, retirados e restaurados sem excluir o conteúdo original.
5. A API gera uma nova `ReportVersion` ; visualizar e baixar usam o arquivo privado.

Antes da primeira publicação, a gerente pode reabrir a seleção ou cancelar o rascunho. Depois que há versão publicada, essas ações destrutivas ficam bloqueadas para preservar o histórico.

Estrutura visual do PDF

- **Página 1:** leitura executiva, total de iniciativas, áreas mobilizadas, evidências, até três destaques, até três pontos de atenção e três prioridades.
- **Páginas seguintes:** síntese e cards detalhados agrupados por área.
- **Versões:** numeração sequencial, autor e data de geração, sem sobrescrever o artefato anterior.

Bloqueios antes de publicar

O front impede a geração quando encontra qualquer uma destas pendências:

- leitura executiva vazia ou ausência de cards ativos;
- campos editoriais obrigatórios vazios ou acima dos limites;
- mais de três destaques ou mais de três pontos de atenção;
- destaque sem evidência;
- decisão necessária sem pedido claro;
- próximo passo sem responsável ou prazo.

7. Contrato com a API Django

A integração está ativa por meio de `httpApi` . Os grupos principais do contrato são:

- **Sessão:** saúde, CSRF, login, usuário autenticado e logout.
- **Cadastros:** áreas, usuários e recursos administrativos.
- **Ciclos:** listar, criar, ajustar prazo, encerrar e reabrir.
- **Coleta:** listar/criar/editar relatos, panorama e gestores pendentes.
- **Curadoria:** gerar rascunho, editar relatório/seções/cards, ordenar, retirar e restaurar.
- **Publicação:** listar versões, gerar PDF e obter URL autenticada.

Listagens paginadas são percorridas pelo helper `listAll` . Uploads usam `FormData` ; os demais comandos enviam JSON. Mensagens de validação e domínio retornadas pelo Django são exibidas ao usuário, inclusive erros de permissão, conflito e estado do ciclo.

8. Administração

A área administrativa é dirigida pelos metadados devolvidos por `/administration/resources` . Assim, campos, colunas e capacidades de criar, editar e excluir respeitam o backend.

- CRUD de usuários, áreas e ciclos;
- grupos e permissões técnicas para superusuários;
- consulta de relatos, fotos, relatórios, seções, cards e versões;
- busca, paginação, detalhes, arquivos vinculados e trilha de auditoria;
- confirmação para exclusões e apresentação de conflitos de integridade.

Grupos e permissões técnicas atendem ao Django Admin. O perfil de negócio (`gestor` , `gerente` ou `admin`) continua governando os fluxos do synthesis.

9. Validação e testes

Resultado verificado em 14/09/2026:

Comando	Resultado	Escopo
<code>npm run typecheck</code>	Aprovado	Tipos TypeScript
<code>npm run lint</code>	Aprovado	Regras ESLint
<code>npm test</code>	22/22 aprovados	3 arquivos Vitest

As suítes automatizadas disponíveis são:

- `src/test/domain.test.ts` : domínio, perfis, ciclos, cards, versões e validação editorial;
- `src/test/auth.test.ts` : sessão e comportamento de autenticação;
- `src/test/administration.test.tsx` : CRUD, busca, capacidades e erros administrativos;
- `npm run test:e2e` : jornada Playwright com API mockada;
- `npm run test:e2e:integration` : jornada real com Django e PostgreSQL efêmero.

O E2E integrado cobre sessão/CSRF, upload válido e inválido, edição própria, isolamento de objetos, curadoria, ordenação, retirada/restauração, duas versões reais de PDF, fechamento do ciclo, proteção de arquivos, CRUD administrativo e auditoria.

Não executado nesta atualização: build e E2E. Eles exigem mais tempo e, no caso do E2E integrado, Docker/PostgreSQL e ambiente de aceite preparado.

10. Implantação e operação

- Node 22 é a versão de referência registrada em `.nvmrc`.
- O build usa `next build --webpack` e produz artefato standalone.
- Em produção, frontend e API devem compartilhar a mesma origem HTTPS.
- O proxy reverso encaminha `/api` e os arquivos privados ao backend.
- A distribuição do MVP é manual: baixar o PDF versionado e anexá-lo ao canal institucional.

11. Arquivos de referência

- `src/services/api.ts` : contrato e implementações HTTP/mock.
- `src/services/auth.ts` : sessão e proteção CSRF.
- `src/types.ts` : entidades compartilhadas.
- `src/components/activity/activityTemplates.ts` : modelos e limites editoriais.
- `src/components/report/reportValidation.ts` : bloqueios de publicação.
- `src/components/report/ExecutiveReportPage.tsx` : primeira página do briefing.
- `src/app/relatorio-semanal/[id]/editar/page.tsx` : orquestração do editor.
- `src/services/administration.ts` : contrato administrativo.
- `e2e/integration.spec.ts` : aceite integrado do MVP.

12. Regras de manutenção

- Preservar a separação entre relato original e snapshot editorial.
- Centralizar chamadas HTTP nos serviços, sem `fetch` espalhado por componentes.
- Tratar `RoleGuard` como autorização visual, nunca como barreira de segurança.

- Manter mensagens de domínio do backend visíveis e acionáveis.
- Não sobrescrever versões já publicadas nem expor arquivos sem autenticação.
- Executar typecheck, lint, testes, E2E e build antes de uma entrega de produção.

Estado atual: o frontend deixou de ser apenas um protótipo com mocks. Ele opera sobre a API Django, implementa os três perfis do MVP e mantém o mock como apoio para testes isolados.